

Airport Modelling Project

Group 41: Planning and Design

1 Planning

1.1 Methodology

An Agile-inspired development methodology was adopted for this project. It allows for flexible and collaborative working which supports incremental progress given the the technical complexity of the airport modelling system. The sprint-based structure enables the team to adjust priorities based on technical discoveries and continuous understanding of requirements. This is imperative for this simulation project due to the complexity of interactions between different components. The true nature of the complexity may only appear during the implementation, so remaining flexible is vital. The client specification includes optional features which can be implemented after the core functionality is met, so the project benefits from iterative development over a fixed approach. Each sprint delivers a working increment that can validated and tested early. This allows for continuous integration testing rather than just an end of project test which could overlook serious issues.

Agile was selected over a traditional Waterfall methodology, which would not facilitate the rapid, iterative development suited to this project. Also, the waterfall methodology's late-stage testing approach creates significant integration risk. The client strongly emphasised producing "a useable and reliable solution which meets the key objectives, rather than feature-rich code which does not work" which aligns better with an Agile methodology. It emphasises cross-functional teams and collective code ownership which increases resilience to delays and unforeseen issues. Overall, this methodology provides sufficient structure that ensures steady progress and accountability alongside the flexibility required to accommodate technical demands, time constraints and potential changes in customer expectations and requirements throughout the project lifecycle.

1.2 Risk Management

Risk ID	Risk Description	Inherent Risk	Mitigation Strategies	Residual Risk
R1 - Project Risk	Team members may become unavailable due to illness, timetable conflicts or reduced participation, leading to delays in development.	Serious	Avoidance: Clearly define roles and responsibilities early in the project. Minimisation: Use Git and regular meetings to ensure progress visibility. Contingency: Ensure key components can be taken over by another team member if required.	Tolerable
R2 - Project Risk	The size and complexity of the airport simulation system may be underestimated, resulting in increased time pressure in later stages.	Serious	Avoidance: Prioritise must-have requirements during planning. Minimisation: Adopt an incremental development approach focusing on core functionality. Contingency: Reduce the complexity of non-essential features if time becomes limited.	Tolerable
R3 - Product Risk	Misinterpretation or changes in requirements may lead to incorrect system behaviour.	Serious	Avoidance: Strictly follow the Requirements Analysis Document. Minimisation: Isolate requirement-dependent logic within the model layer. Contingency: Prioritise safety-critical and core scheduling rules.	Tolerable
R4 - Product Risk	Development tools or frameworks may not perform as expected.	Tolerable	Avoidance: Select familiar and course-recommended tools. Minimisation: Keep the system design simple and modular.	Insignificant
R5 - Business Risk	Changes in technology or implementation approach may require redesign.	Tolerable	Avoidance: Use a modular architecture. Minimisation: Separate core logic from UI components.	Insignificant
R6 - Business Risk	The final system may not fully align with customer or assessment expectations.	Serious	Avoidance: Align development with coursework requirements. Minimisation: Refer to and optimise requirements during development. Contingency: Justify design trade-offs clearly.	Tolerable

Table 1: Project risk register, identifying potential technical and operational disruptions with associated mitigation strategies.

1.3 Development Timeline

The following Gantt chart details how the project will be broken down into sub-tasks and how they will be assigned across the 3 sprints. Note that the week numberings match the University term timetable for developer convenience.

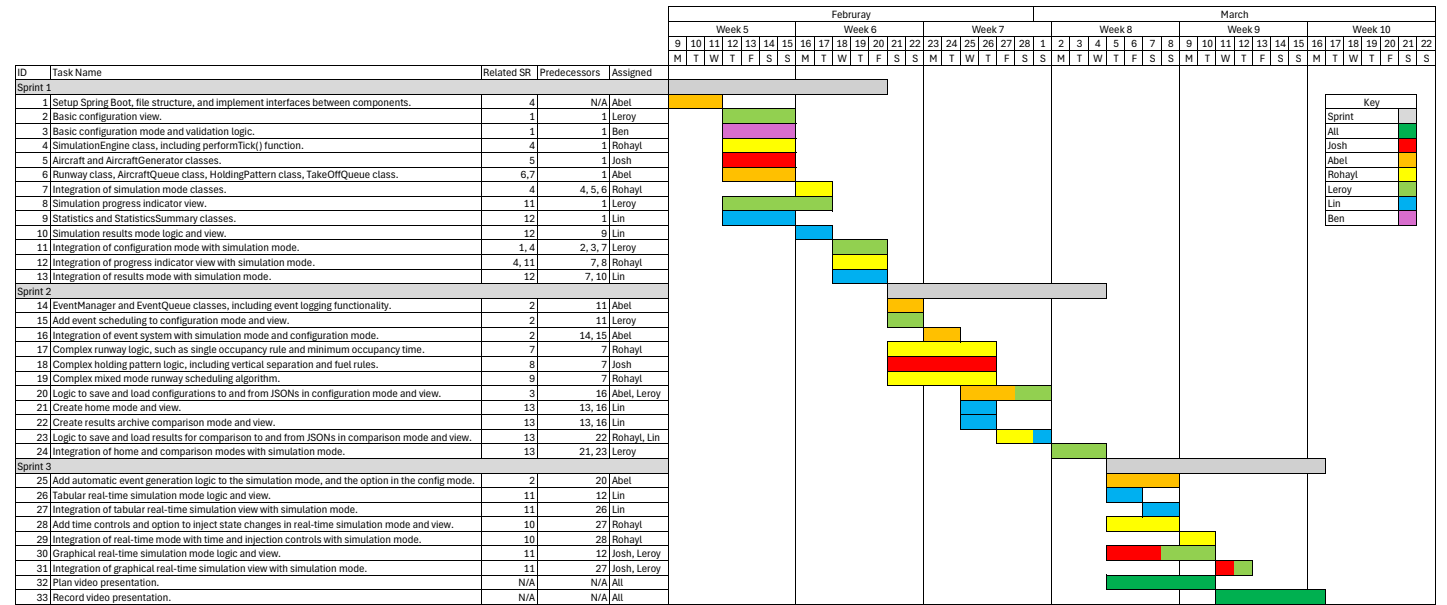


Figure 1: Gantt chart, showing the elements of each sprint and what task each member is assigned.

Since we are following an agile-inspired methodology, this plan is not rigid. Therefore, the above Gantt chart will be adjusted during the project lifecycle in response to any delays, technical challenges, or changing customer requirements which may impact the scheduled tasks. Similarly, if team members become less available during the project, then tasks may have to be reorganised or redistributed to ensure the most important functionality is delivered before the project deadline. Another point to note with the above chart is that only one task has been explicitly assigned to Ben, the Software Tester. This is because his role is integrated across all project tasks, including writing unit tests before components are implemented to facilitate test-driven development. He will also be heavily involved in the integration tasks listed, performing system and integration tests to ensure the components of the system interact correctly to satisfy the customer's requirements.

The timeline of the project was determined in a methodical way. A key part of the first task, and subsequent closely-linked tasks, is to implement the interfaces between components. This is because it allows components to be developed in parallel, while ensuring they can communicate successfully. By considering the integration of components at the start of each development phase, it will make the integration process easier, and help to reduce the chance of delays caused by this process. The timeline itself is organised into three 12-day sprints, evenly dividing the development time available into three periods. Maintaining this consistent sprint length is beneficial as it establishes a predictable cadence and allows the team to accurately measure velocity, ensuring that progress remains transparent and deadlines are met.

The first sprint focuses on implementing the basic simulation lifecycle. This includes a basic configuration mode, the simple logic for the simulation, and a basic outputs view. If all tasks are completed, the sprint will result in a functional system that allows the user to set simulation inputs and view the outputs. The second sprint adds the event system to the program, meaning events can be scheduled, which meets the key user requirement of being able to analyse the impact of such events on the throughput and delay times of airports. Additionally, it adds the more complex simulation logic and rules defined in the specification. With these additions the product would meet all the main client requirements. To make the system even more suited to the client, this sprint will also add the ability to save and load configurations and results for comparison. Finally, the third sprint focuses on adding optional features. This includes automatic event generation and real-time simulation views. These features will benefit the user, however they are far less critical to the customer's core requirements which is why they are deferred to the final sprint.

Due to this timeline, the plan is inherently flexible. This is because the plan aims to implement all core client requirements by halfway through Sprint 2. This provides a buffer within the schedule to ensure all essential functionality is finalised even if unforeseen development delays occur. Similarly, at the end of the project, if some features are unable to be finished in time, this structure ensures these will be the optional features and not core requirements. This flexibility is also important because the main difference between our methodology and a true Agile methodology is that we have limited customer communication. Therefore, when we have the client consultation, the requirements may change. As a result, it is important to have a flexible plan to ensure it can accommodate these changes and deliver a system that meets the client's true needs by the deadline.

2 Design

2.1 System Architecture

The system follows the Model-View-Controller architectural pattern. This pattern separates presentation and interaction from system data by structuring the system into three logical components that interact with each other [1]. The Model runs the simulation and manages the data from the previous runs. The View manages how the data is presented to the user, providing the input and output screens and the real-time simulation screens. The Controller manages user interaction, e.g. passing the simulation input parameters from the Input View to the Model.

One benefit of this pattern is that the components are isolated and therefore modular. This facilitates parallel development since it makes it easier for a developer to understand and modify each particular unit, regardless of the implementation and knowledge other units [2]. This separation of components supports test driven development, as each component can be developed and tested individually. This is done before performing integration testing when testing component interaction [3]. Another benefit is that it supports presentation of the same data in different ways, changes made in one representation would show in all of them. For example, the real-time map view could be complemented with another view showing a table of fuel remaining for each plane. Finally, the separation of components allows extensibility because new features can be added to components without requiring huge changes to other modules. This is a significant benefit because it makes development more flexible, changes to customer requirements can be more easily accommodated.

While the MVC pattern provides significant structural advantages, it introduces additional architectural complexity through the decoupling of components. This can lead to increased development overhead, as even simple updates may require synchronised changes across the Model, View and Controller to maintain consistency. However, this complexity is justified for the airport simulation because the system's logic, detailed in the Requirements Analysis document, is non-trivial and is well-suited to the rigorous separation of concerns that MVC provides. By accepting this initial abstraction overhead, the software will be developed in a more organised way, ensuring the final system remains maintainable and robust as functionality is added to the system during the iterative development process.

The following diagram shows the design of the system using the MVC pattern. The smaller boxes demonstrate the subsystems of each of the three components, while the arrows show how data/control is passed between them. Note that the word "state" refers to the current state of the system, for example waiting for user input or running the simulation. A query is when the Controller requests information from the Model, for example information about a particular plane in the model or the result of previous runs.

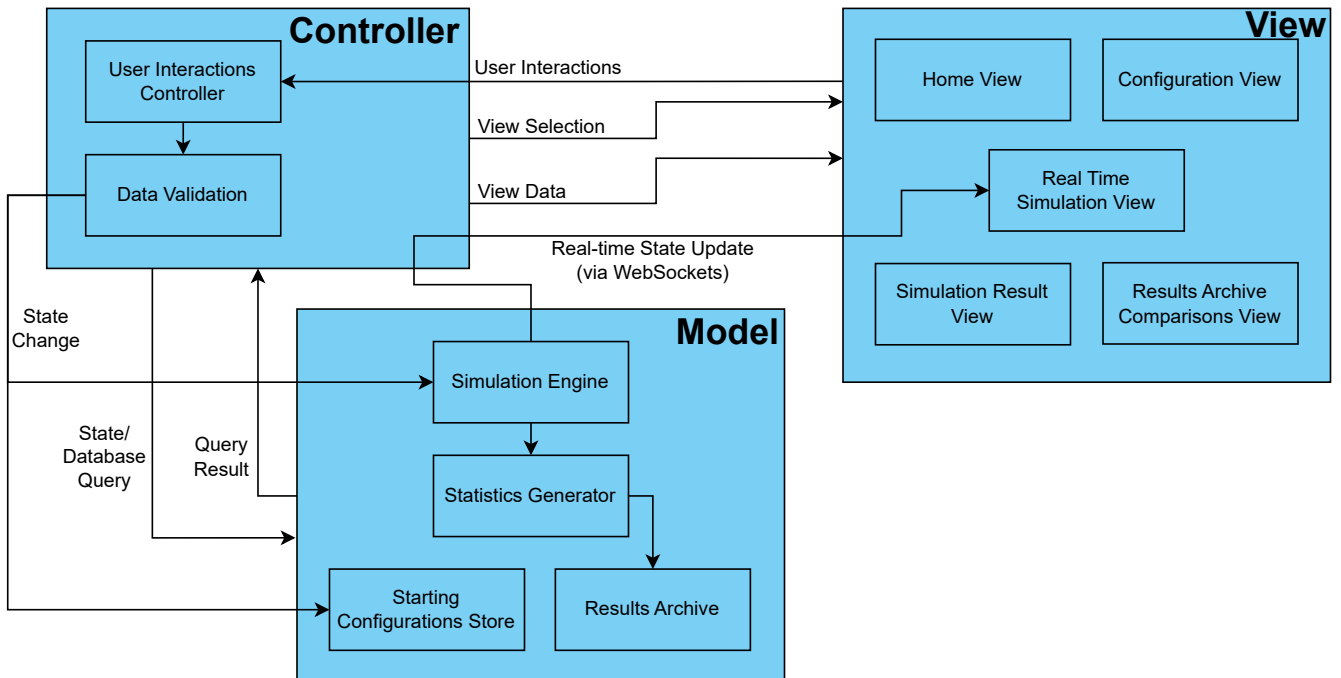


Figure 2: MVC architectural pattern diagram, showing interactions between systems and subsystems.

2.2 Component Design

2.2.1 Home Mode

The Home Mode is shown in the diagram in the System Operational Modes section of our Requirements Analysis document. It is the entry screen to our application and allows the user to:

- Start a new simulation
- Navigate to the results archive and compare previous simulation runs
- Quit the application

2.2.2 Configuration Mode

After the user wants to start a new simulation from the Home Mode, they will enter the Configuration Mode. The following finite state diagram depicts the state flow.

This mode allows the user to:

- Configure the simulation settings.
- Load a previous configuration.
- Start the simulation if the configuration is valid.

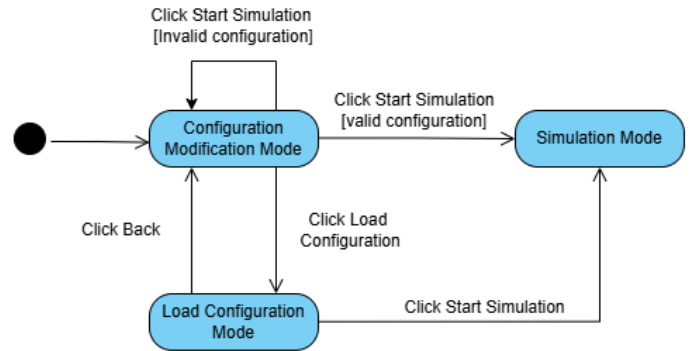


Figure 3: State Diagram for Configuration Mode.

Load Configuration

Configuration Page

Simulation Mode: Real Time

Start Simulation

Input Parameters

Number of Runways

Max Limit: 10

- 10 +

Inbound Rate /hr

Range: 0 to 100

15

Outbound Rate \hr

Range: 0 to 100

Value

Simulation Duration (minutes)

Range: 60 to 1440

Value

Mechanical Failure Rate

Range: 0.00 to 0.10

Value

Passenger Health Issue Rate

Range: 0.0 to 0.1

Value

Runway Inspection Rate

Range: 0.0 to 0.1

Value

Snow Clearance Rate

Range: 0.0 to 0.1

Value

Equipment Failure Rate

Range: 0.0 to 0.1

Value

Max Delat Time (minutes)

Max Limit: 60

Value

Runways

	Status	Mode
Runway 01	Available	Mixed
Runway 02	Available	Mixed
Runway 03	Available	Landing
Runway 04	Snow Clearance	Take-Off
Runway 05	Runway Inspection	Take-Off
Runway 06	Available	Mixed
Runway 07	Available	Mixed
Runway 08	Available	Landing
Runway 09	Equipment Failure	Take-Off
Runway 10	Available	Take-Off

Scheduled Events

Events	Time (into simulation)	Duration
Equipment Failure on Runway 01	100 minutes	50 minutes
Passenger Health in Holding Pattern	120 minutes	Indefinite
Snow Clearance on Runway 06	150 minutes	60 minutes
Runway 02 Mode Set to Take-Off	250 minutes	Indefinite

Schedule New Event

New Event:

Event Type	Event	Location	Time	Duration
Runway	Inspection	Runway 07	150 mins	120 mins

Figure 4: Webpage Design of Configuration Page.

In the above design, the ranges and maximum limit for input boxes are displayed to the user. If the user input is invalid, an error message would appear. To ensure data integrity and minimise user error, drop-down menus are used for parameters with discrete ranges. This restricts input to valid selections and provides clear visibility of all available options. Otherwise, input boxes are used instead. For the general colour scheme for menus, we chose to design dark text on a light background, because this has been shown to lead to better legibility. [4]

2.2.3 Simulation Mode

Once the user has configured their simulation settings, they will enter the Simulation Mode where the simulation would take place. Since this project lends itself to an object-oriented approach, the following figure illustrates the fundamental class diagram consisting of all the core components of the model and the associations between classes.

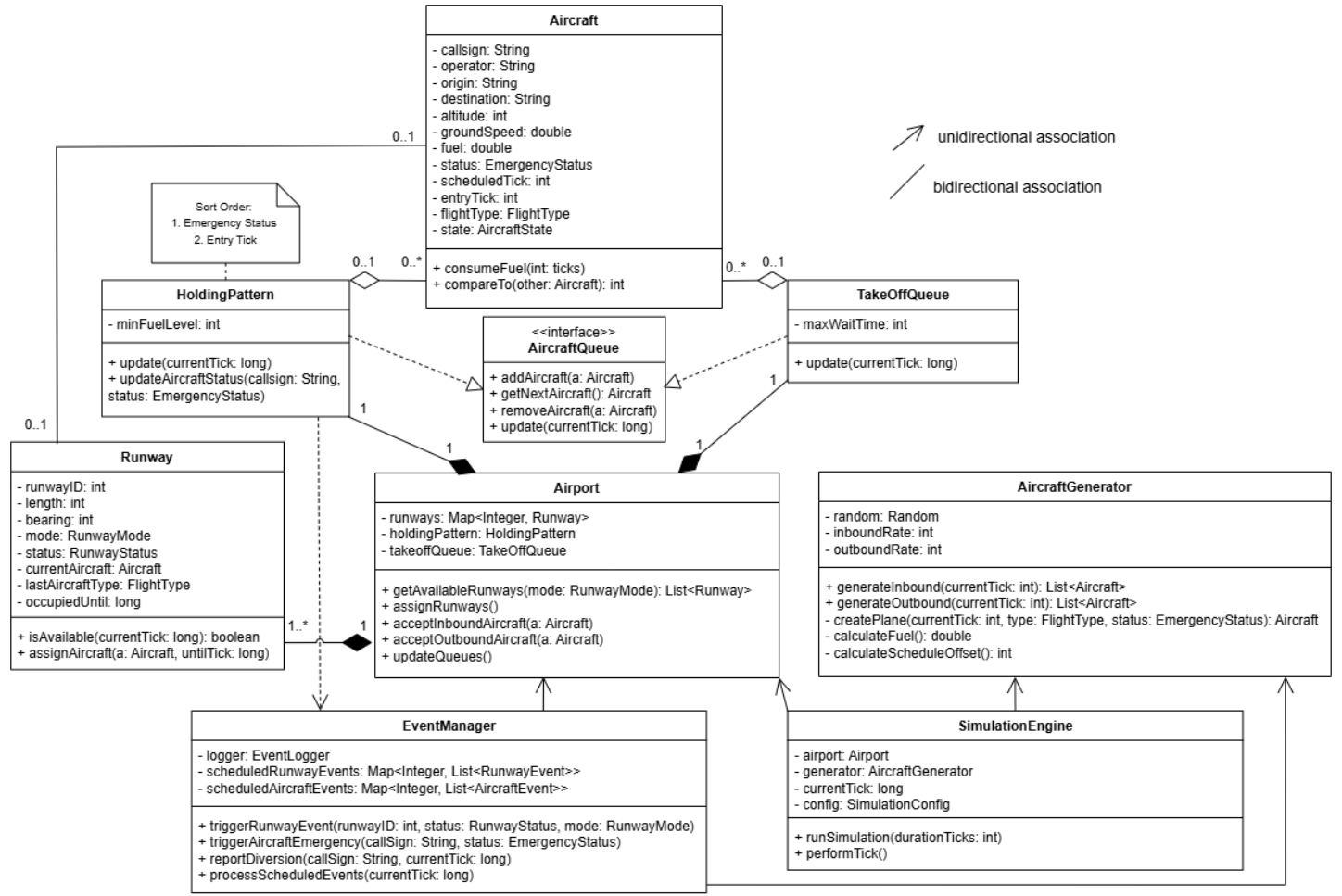


Figure 5: Class Diagram for modelling system.

The system is driven by the **SimulationEngine**, the **AircraftGenerator** and the **Airport** classes. Classes such as the **SimulationController**, **Configuration** and **Statistics** classes have been omitted to ensure the diagram focuses on the main parts of the system. The **SimulationController** (depicted in section 2.1.1 as in the MVC diagram) would instantiate the **SimulationEngine** and pass user inputs to the **EventManager**.

This design ensures a natural hierarchy within the system. This is critical as it allows the separation of responsibilities such that no one class is in charge of itself. Instead, select classes ensure the functionality of its own subsystem. To begin with, **SimulationEngine** acts as the master class and controls the discrete time steps of the simulation along with its configuration, aircraft generation and processing. The **Airport** class negotiates the assignment of runways and aircraft to their respective queues. It is polled by the **SimulationEngine** regularly for each tick to run through its routine.

The **Aircraft Queue** was a key design decision to introduce an interface for **HoldingPattern** and **TakeOffQueue** to allow polymorphism and modularity. While both queues update per tick, their logic differs: **TakeOffQueue** monitors wait times for cancellations, whereas **HoldingPattern** tracks fuel levels for diversions (UR-FR-14). Crucially, the **HoldingPattern** implements a priority queue to automatically sort aircraft by emergency status and then arrival time, satisfying UR-FR-16.

The **EventManager** class handles the event subsystem. This includes the scheduling and logging of events to allow user comparisons with results from other simulation runs. This class is dependant on the user configurations since it manages their scheduled events (UR-FR-6). It orchestrates the **AircraftGenerator** class when it aligns the current tick with an event in its event maps. A dependency relation exists with the holding pattern because fuel diversions are emergency events which should be notified to the **EventManager** (UR-FR-12). Furthermore, the user is allowed to change the emergency statuses of aircraft within the holding pattern through the user interface (UR-FR-29). This requires controller interaction with the **EventManager**, although this is not depicted on the diagram to aid abstraction down to the core modelling system.

Note: Enumerations were also omitted from Figure 5 since these are self explanatory.

The proposed design will use a tick system where one tick represents a unit of time within the simulation. Each process will be synchronous to prevent race conditions. The below figure represents a sequence diagram for the `performTick` function within the `SimulationEngine`. It illustrates the dynamic view of the core simulation loop.

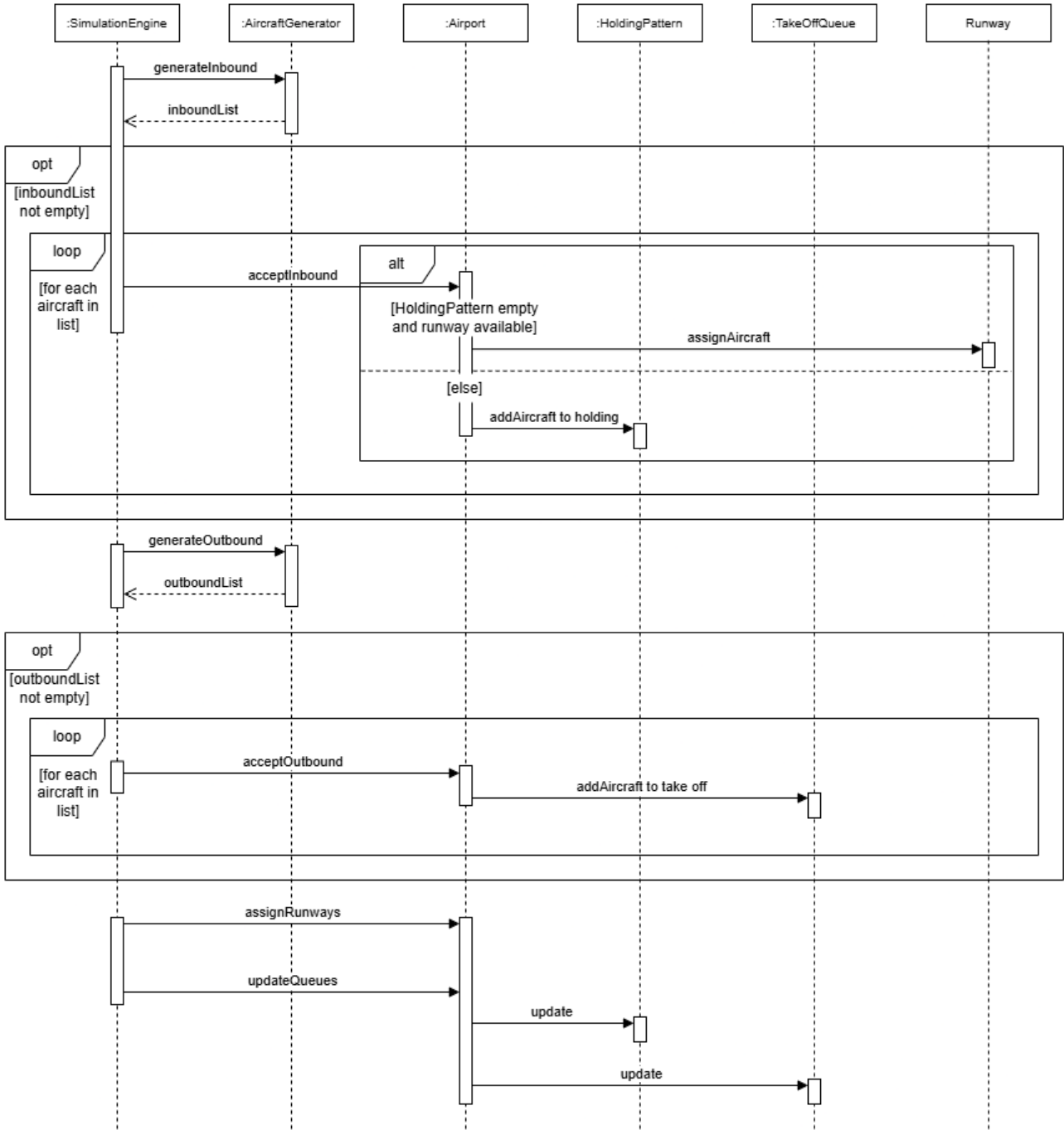


Figure 6: Sequence Diagram depicting one unit of time (tick).

The simulation cycle begins with the `SimulationEngine` polling the `AircraftGenerator` for a list of new inbound aircraft, allowing for flexible throughput. These are passed to the `Airport` and delegated to the `HoldingPattern`. To satisfy UR-FR-13, the arrival aircraft will be assigned to an available runway if the `HoldingPattern` is empty. This is not done for outbound aircraft since they have to be cleared for take-off (as per UR-FR-18), so all outbound enter `TakeOffQueue`.

Once all aircraft are queued, the `Airport` assigns eligible aircraft to available runways. Performing assignment after queuing ensures the algorithm considers the complete, up-to-date state of the airspace, enhancing system robustness.

Finally, the `Airport` triggers updates to both the `HoldingPattern` and `TakeOffQueue` in order for them to re-prioritise, consume fuel and check for diversions and cancellations (further description of these `update` functions lie within the Simulation Mode section of this report). The scheduling of these calls at the *end* of the tick ensures the aircraft that have just arrived do not suffer fuel penalties before having a chance to be assigned to runways, this will help prevent future logic errors.

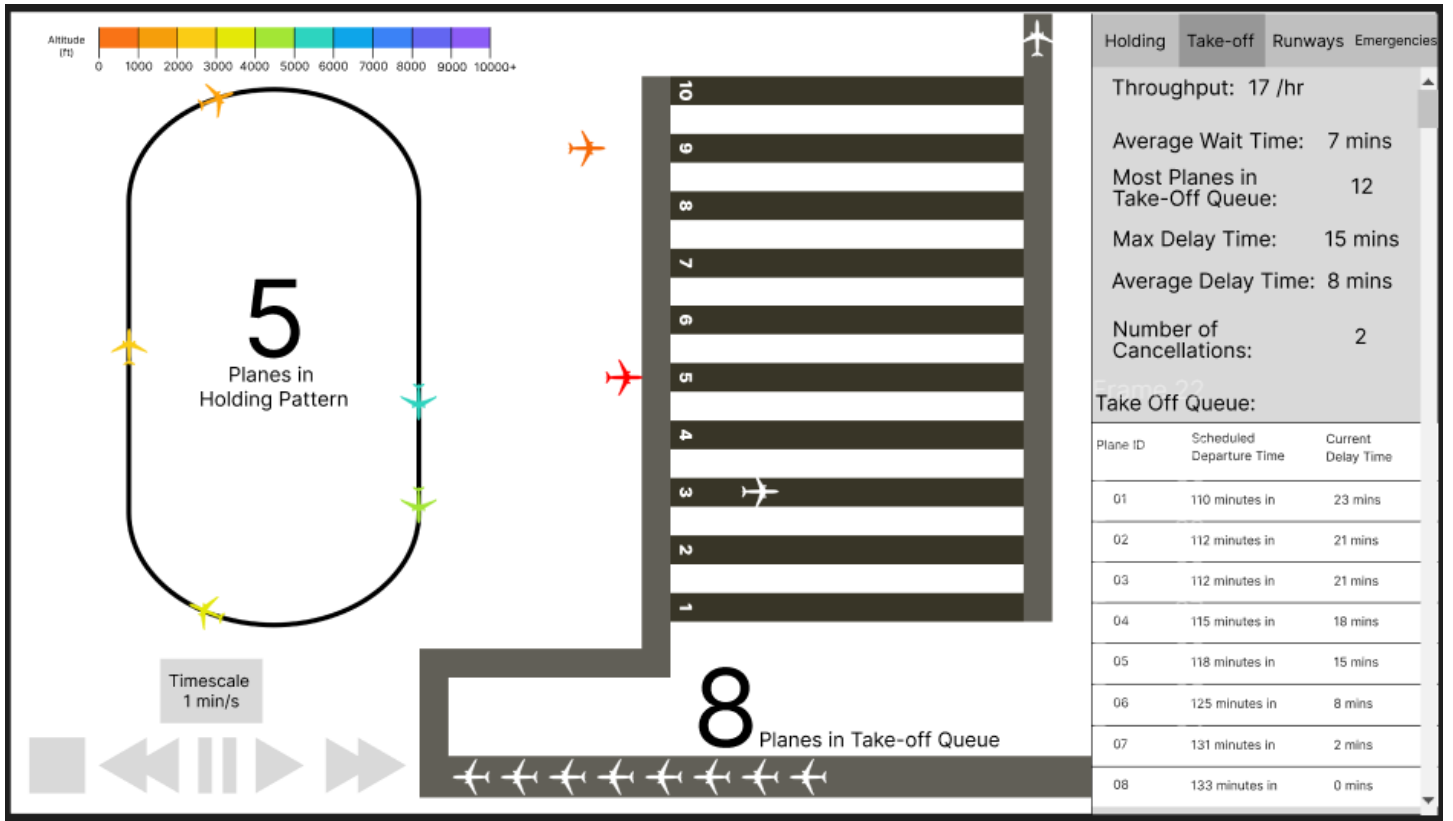


Figure 7: Webpage Design of Simulation View.

In the design of the simulation view, new planes will spawn from the top left in the sky or the bottom right side of the take-off queue. There is also a timescale box that shows the user the simulation speed, which updates as they press the buttons below it (to reduce or increase simulation speed, or pause simulation). The user can end the simulation by clicking the stop button in the bottom left.

The running values of data points can be viewed in the side window. The main contents of the Take-off tab can be seen. In Holding tab, there will be a table representation of the holding queue, along with key data points. In the Runways tab, the user should be able to edit the runway status and modes. In the Emergencies tab, information about current emergencies can be viewed (e.g. how long current emergency has gone on for). The side window could be what is shown in a different simulation mode, slightly expanded with the potential to show more running data points. This simulation mode would satisfy UR-FR-25. The user should be able to click on a plane to view some individual information in a box that follows the plane. The user should also be able to start an emergency event on a specific plane too.

The altitude of planes is shown through colours according to the scale. This scale intentionally avoids the colour red, since that is reserved for planes with emergencies. To further distinguish themselves, emergency planes would flash red too. Planes on the ground will be coloured as white. Following this design should satisfy requirements UR-FR-24 to UR-FR-29.

2.2.4 Simulation Results Mode

When the user finishes running their simulation, they will be greeted with the results of their run. The following finite state diagram depicts the flow.

This mode allows the user to:

- Save the simulation results - this saves the configuration, the statistic results and the event log of the simulation
- Return to the Home Mode (discards the simulation if not saved)
- Compare the simulation with other results (redirects to the Results Archive Comparison Mode)

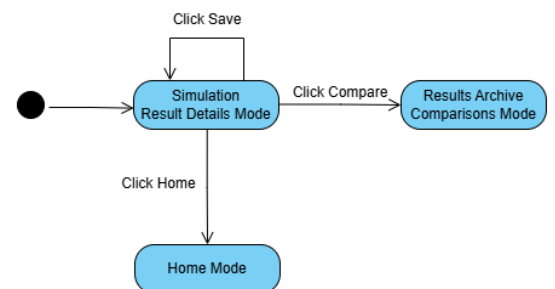


Figure 8: State Diagram for Simulation Results Mode.

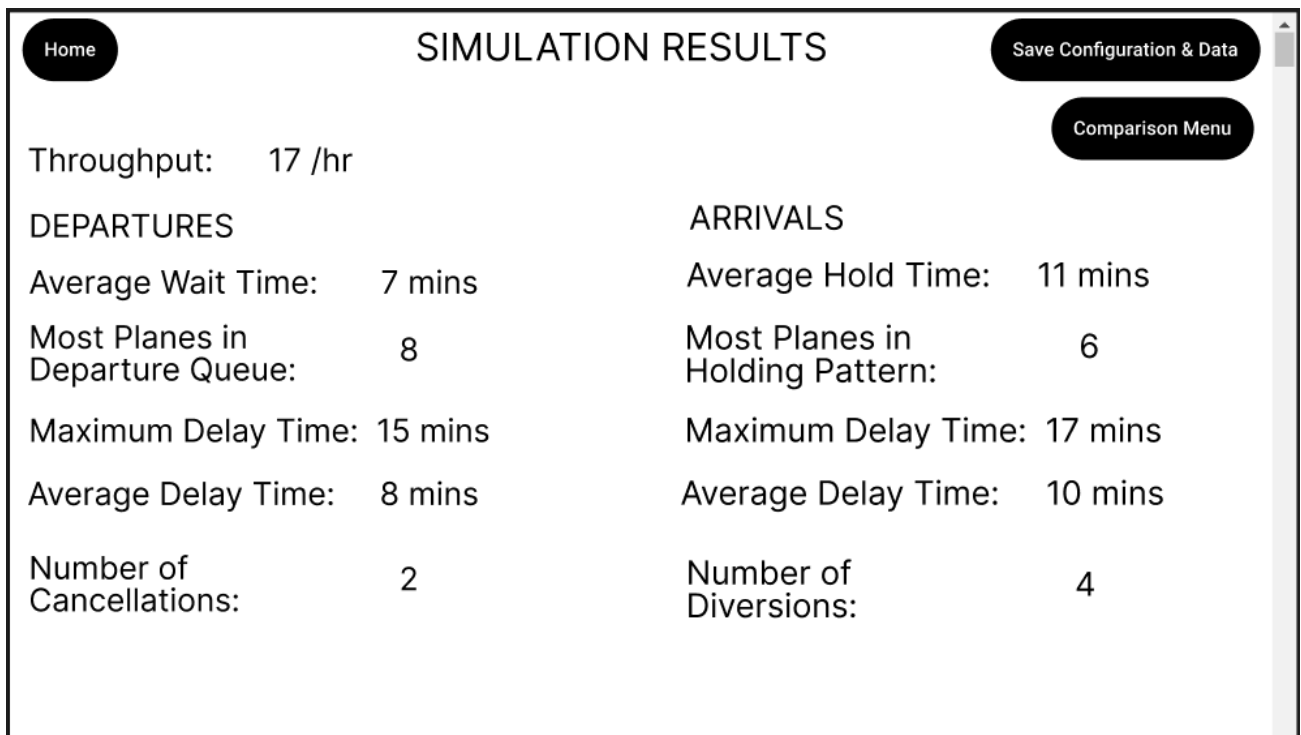


Figure 9: Webpage Design of Simulation Results.

In this design, the throughput metric is at the top, as it is the key metric in the Problem Statement. Departure information is listed as the top priority in the Output section of the specification. Since the user is expected to read from left to right, departure information is on the left of the screen, with arrival information on the right.

2.2.5 Results Archive Comparison Mode

This comparison mode can be seen on the diagram in the "System Operational Modes" section of our Requirements Analysis document. This mode can be accessed through the home screen and final screen of the simulation. When the user enters this mode, it allows them to:

- Compare two simulations in the results archive
- Return to the Home Mode

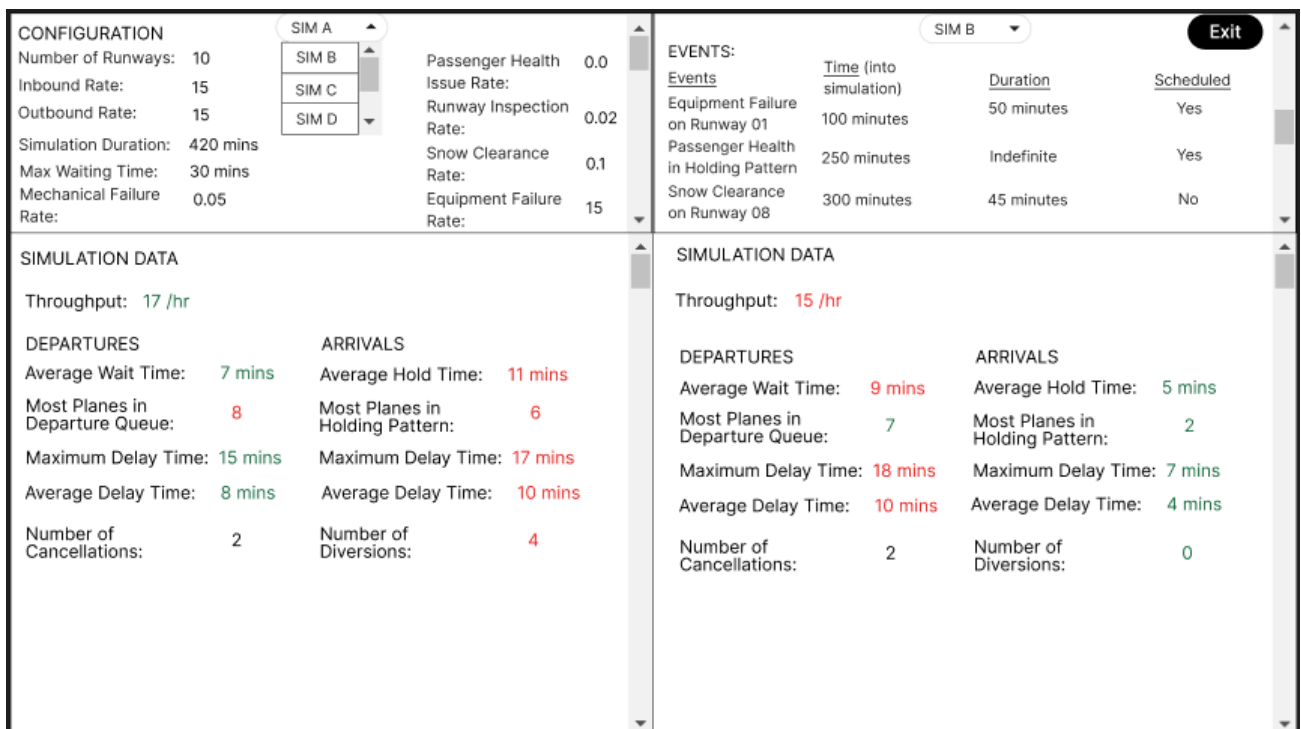


Figure 10: Webpage Design of Comparison Menu.

In this design, the user can select which saved simulation to compare using the dropdown boxes. The configuration information for each simulation is included at the top. The simulation data should match what can be seen on the results page, with green text showing a ‘better’ value than the other simulation in that metric. The exit button allows the user to go back to the home page.

2.3 Technology Implementation

2.3.1 Technology Stack Summary

Component	Selected Technology	Primary Justification
Backend	Java 25	Object-Oriented nature maps to the domain model. Strong type safety is necessary ensures robustness.
Application Framework	Spring Boot	Enables backend and frontend communication via RESTful APIs.
Frontend	HTML5 / CSS / JS	Provides an accessible, commercial-standard interface.
Data Persistence	JSON (Jackson Library)	Human-readable format for configuration and logs, satisfying the requirement for user-editable inputs.
Build & Dependencies	Maven	Automates dependency management (Spring, Jackson, JUnit) and simplifies the build process.

2.3.2 Backend (Application logic)

Java 25: We selected Java as our core language for the simulation. This is because the project requirements involve complex entities and relationships that interact hierarchically. Java’s strict Object-Oriented Programming (OOP) structure enables us to model these explicitly, through inheritance, polymorphism and interfaces. Our market research led to initial consideration of Python, but its dynamic typing can lead to obscure runtime errors. Java’s strict static typing is critical in our modelling systems since it catches errors at compile time rather than runtime. Furthermore, given our teams familiarity with Java already, it was easily the superior language choice, especially when using the latest stable release (Java 25).

Spring Boot: In order to build a "commercial" software solution, an application framework is necessary. Spring Boot is our choice since it integrates Java seamlessly into web applications and accelerates production [5]. It embeds a web server directly within our application to decouple the front and back end (resulting in separation of concerns) without having to rely on external web servers [5]. Therefore, an application can run on any platform simply and easily. Whilst it may be possible for us to create a web server using solely Java, it is difficult and not recommended. Spring Boot is necessary considering the team’s lack of experience to simplify the server creation process. Although we considered the use of **JavaFX** for a desktop experience, it is less representative of modern web-based commercial software and does not offer the flexibility and future-proofing that Spring Boot has.

2.3.3 Frontend (User Interface)

HTML, CSS & Javascript: These technologies are fundamental for any intuitive web-based interface. Visual representations can use HTML5 Canvas API since this is specifically designed for high performance rendering, so it is ideal for drawing aircraft positions in simulation real time [6]. Javascript will act as the client, interacting with the Spring Boot backend for the latest data and updating the Canvas accordingly.

2.3.4 Data Persistence (Storage)

JSON & Jackson: The specification requires the system to accept user inputs, output results, and allow the user to compare simulation runs (UR-FR-8 & UR-FR-34). As a result, the saving and storage of data is crucial for this application. Initially, the team considered using an SQL based technology like PostgreSQL in order to create database storage for the application. However, it quickly became clear that implementation of a complex relational database without the need for any user-profile based system was highly unnecessary, the data was flat and hierarchical as opposed to relational. Simple file storage through JSON was found as the ideal choice, it allows a human-readable and lightweight platform for data persistence. Unlike Javascript, Java does not have native support for JSON serialisation, so a translator such as the Jackson library [7] is required to ensure robustness and remove the need for complex parsing logic in order to store and access JSON files. Rather than the need for a separate database, the user’s own file system is suitable for data persistence.

2.4 Testing and Validation Strategy

The testing and validation strategy ensures that the system behaves as described in the client specification and produces correct and consistent simulation results. Testing focuses on checking both individual parts of the system and how these parts work together during a simulation run.

Testing is designed to be traceable to the system requirements defined in Section 4 of the Requirements Analysis document. Each system requirement is associated with one or more tests, which are listed in the System Requirements tables to show how correct behaviour is verified.

Unit testing is used to verify core simulation logic in isolation, such as landing priority rules, fuel usage and diversion thresholds, queue behaviour, cancellation conditions, and runway occupancy constraints. Boundary cases, including fuel reaching 10 minutes remaining and queues reaching maximum capacity, are explicitly tested to ensure correct behaviour at limits. Unit tests will be written for components before implementation commences and will be implemented using JUnit 6 (for Java 25). This facilitates test-driven development which helps developers create more reliable, maintainable code, and ensures the software meets the specific requirements from the outset.

Integration testing is used to confirm that different parts of the system work correctly together. This includes verifying that configuration inputs are correctly applied to the simulation model, that changes to runway status or mode propagate correctly during runtime, and that mixed-mode runway behaviour follows the defined rules. Examples of both unit and integration tests are provided in the System Requirements tables. Crucially, integration testing occurs throughout the project lifecycle to allow sufficient time to resolve any technical problems between components before the deadline. We will utilise Spring Boot Test with MockMvc and Mockito to ensure correct and consistent behaviour by confirming HTTP requests trigger the desired state changes in the model when received by Controller endpoints. These libraries allow testing without the need for a live server environment.

System testing is used to validate complete simulation runs under realistic scenarios, such as high traffic levels, runway closures, and emergency landings. These tests ensure that overall system behaviour remains consistent and that outputs such as delays, cancellations, and diversions are correctly calculated.

Formal user acceptance testing could not be performed due to the absence of direct access to the client. To partially address this, the system was reviewed by individuals outside the development group to confirm that the interface was understandable and that configuration and simulation controls were clear to use. While this does not replace full client feedback, it provides basic validation of usability and interaction flow.

3 References

- [1] Ian Sommerville. *Software Engineering*. 10th Global Edition. Boston: Pearson, 2016. ISBN: 9781292096148.
- [2] Glenn E Krasner, Stephen T Pope, et al. “A description of the model-view-controller user interface paradigm in the smalltalk-80 system”. In: *Journal of object oriented programming* 1.3 (1988), pp. 26–49.
- [3] Abdul Majeed and Ibtisam Rauf. “MVC architecture: a detailed insight to the modern web applications development”. In: *Peer Review Journal of Solar & Photoenergy Systems* 1.1 (2018), pp. 1–7.
- [4] Cosima Piepenbrock et al. “Positive display polarity is advantageous for both younger and older adults”. In: *Ergonomics* 56.7 (2013), pp. 1116–1124.
- [5] VMware, Inc. *Spring Boot Reference Documentation. Developing Web Applications*. Version 3.2.5. Spring. 2024. URL: <https://docs.spring.io/spring-boot/docs/3.2.5/reference/htmlsingle/#web> (visited on 02/08/2026).
- [6] MDN Web Docs. *Canvas API*. Documentation for the HTML5 Canvas element and 2D drawing context. Mozilla. 2026. URL: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API (visited on 02/08/2026).
- [7] FasterXML, LLC. *Jackson Project Home*. Standard JSON library for Java (aka JSON for Java). GitHub. 2026. URL: <https://github.com/FasterXML/jackson> (visited on 02/08/2026).